

Dyia QA — Retest Response (Round 2)

Prepared for: QA — Hanna
Date: **April 2026**
Source brief: *Dyia QA Test Report 3* (Apr 2026)
Scope: items still failing/partial after Round 1 + new BUG-031
Branch: `fix/qa-retest-2026-04-round2`

Why this round is different

Round 1 shipped surface-level patches that QA correctly identified as still-broken. This round goes after the underlying root causes — animation keyframes that trap container transforms, a Stripe webhook that silently no-ops on metadata mismatch, an OpenAI Responses API thread that never receives confirm output, and a clipboard flow that loses user activation after every `await fetch`. Each fix is paired with the actual file & line where the cause lived, so future regressions can be triaged from this report alone.

Fixed Root-cause fix shipped & verified

Fixed (code) Shipped; needs prod env to verify

Needs QA Verify on real Stripe/Clerk session

Partial Mitigated; doc gap noted

Blocked External dependency

Round 2 results — items that QA marked Still Present / Partial / new

Bug ID	Description	Severity	Area	Result
BUG-011 / AI-003	Ask Dyia AI cannot update a job it just logged (works for quote→job conversion only).	Major	Ask Dyia / Insights	Fixed
BUG-015	Source & Revenue sort dropdown text truncated on desktop after Round 1 fix.	Cosmetic	Jobs	Fixed

Bug ID	Description	Severity	Area	Result
BUG-016	Log Daily Expenses / Convert-to-Job popups offscreen on long lists.	Minor	Jobs / Quotes	Fixed
BUG-018	Notification banner cut off on mobile after Round 1 fix.	Cosmetic	Mobile / Home	Fixed
BUG-020	Dyia logo unreadable in dark mode on mobile (Round 1 comment claimed fix but code never updated).	Cosmetic	Mobile / Header	Fixed
BUG-022	Plan shown as "Free" instead of "Basic"; "Try Pro for free" still shown after trial consumed.	Major	Settings / Banner	Fixed (code)
BUG-031 (NEW)	Stripe payments succeed but never reflected in Dyia (quote/job stays unpaid; /payments empty).	Critical	Quotes / Stripe	Fixed (code)
UX gap	"Request Payment" / "Copy Pay Link" pop-up requires manual copy instead of auto-copy.	Minor	Quotes / Jobs / Payments	Fixed

Detailed root cause & fix per item

BUG-011 / AI-003 — AI cannot update a just-logged job

Root cause: The Round 1 fix appended (`job_id: <uuid>`) to the confirm route's success message and stored it in our DB, but `/api/ai/chat/confirm` never calls `responses.create` — it lives outside the OpenAI Responses API thread. The next chat request still sent `previous_response_id` from the *propose* turn, so the model had no way to access the new id from its own context. The fallback ("call `get_user_context` first") also failed because `recentJobs` ordered by `date` only — a brand-new job logged today did not reliably appear at the top of same-day rows.

Fix (3 layers — defense in depth):

- **Client-side carry-over.** `src/components/app/Assistant.tsx` introduces a `lastConfirmedRef` (`{ jobId?, quoteId?, customerName?, date? }`). Set inside both `handleConfirmJob` and `handleConfirmQuote` after confirm, consumed and cleared on the very next outgoing `/api/ai/chat` request body as `lastConfirmed`.
- **Server-side context injection.** `src/app/api/ai/chat/route.ts` builds a `confirmedContextPreamble` when `lastConfirmed` is present and prepends it to *every* input variant (text, vision, attachments), e.g. `[CONTEXT] The user has just saved a job. Use job_id="..."` directly when calling `update_job` — do NOT call `get_user_context` first. This is independent of the OpenAI server-side thread, so the model receives the id even though the confirm step never ran through `responses.create`.

- **Tiebreaker.** `src/lib/openai/handlers.ts` `get_user_context.recentJobs` now orders by `created_at desc` and includes `created_at` in the projection. A brand-new job is always first in the list when the model falls back to lookup.

Verification: build clean, type-checked. Live verification needs a real Clerk session + AI credits — once signed in, log a job via Ask Dya, then in the same thread say "update that job, change revenue to 500" — should succeed without a `get_user_context` round-trip.

BUG-015 — Source / Revenue sort dropdown text truncated on desktop

Root cause: Round 1 swapped `w-full` for `!w-auto`, but native `<select>` with `width:auto` sizes to the *currently selected* option's text, not the widest option. Picking a long label (e.g. "Revenue: High-Low" or a long source name) clipped the chevron padding region.

Fix: `src/components/app/Jobs.tsx` lines ~1581–1611 — Source select gets `min-w-[180px]`, Revenue sort gets `min-w-[200px]`, both now reserve `!pr-10` for the chevron. Text fits the longest option without re-squeezing the search field. Quotes toolbar uses different markup (filter pills) and was not affected.

BUG-016 — "Log Daily Expenses" / "Convert to Job" modals offscreen on long lists

Root cause: A non-trivial CSS finding. `.animate-view-enter` applied to the main content wrapper in `src/app/app/page.tsx` ran the `viewEnter` keyframe with `animation-fill-mode: both`. The 100% keyframe ended at `transform: translateX(0)` — a non-`none` transform value. Per CSS spec, any ancestor with a non-`none` transform becomes the containing block for descendant `position: fixed` elements. So every "fixed" modal in the app (`.modal-overlay`, the inline log-daily-expenses overlay) was positioning relative to the *scrolled* content wrapper, not the viewport. Round 1's `items-start` → `items-center` change moved the panel within that broken coordinate system but could not get it back into the actual viewport on long lists.

Fix: `src/app/globals.css` — `@keyframes viewEnter` now ends at `transform: none` instead of `translateX(0)`. After the animation completes the wrapper has no lingering transform, so `position: fixed` resolves against the viewport again. This single 1-line fix makes *every* centered modal in the app behave correctly on long pages — Log Daily Expenses, Convert to Job, all `.modal-overlay` consumers.

BUG-018 — Trial banner cut off on mobile

Root cause: `TrialBanner` applies inline `style={{ animation: 'bannerSlideDown 0.5s ... both' }}`. The keyframe in `globals.css` ended with `max-height: 80px`. Because of `animation-fill-mode: both`, that 80px `max-height` persisted on the element after the animation finished and won the cascade against Tailwind's `max-h-60`. Combined with `overflow-hidden`, any banner taller than 80px on a narrow viewport (which it is, when CTA wraps to a second row) was clipped — no Tailwind class could fix it.

Fix: `@keyframes bannerSlideDown` end keyframe expanded to `max-height: 16rem` (~256px), with the 60% way-point lifted to `8rem`. Banner can now grow to fit a stacked title + CTA + dismiss row on phones.

BUG-020 — Mobile logo unreadable in dark mode

Root cause: Round 1 added a comment in `TopBar.tsx` claiming `dark:brightness=0 dark:invert` had been removed — but the code on disk *still carried both classes*. The dark-ink wordmark was being put through `brightness=0+invert` on a dark background, producing a low-contrast result. Doc and code disagreed; QA correctly flagged the visible bug.

Fix: `src/components/app/TopBar.tsx` mobile logo now actually drops the dark filters and conditionally swaps to the dedicated `/dyia-logo-dark.png` asset (which exists in `public/`) when `resolvedTheme === 'dark'`. Light theme keeps `/dyia-logo-full.png`. `opacity=90` for both.

BUG-022 — Plan shown as "Free"; "Try Pro free" still after trial consumed

Root cause (two compounded issues):

- The Round 1 Settings card branched to `'FREE'` whenever `uiTier === 'basic'` AND `subscription.status` wasn't `'active' | 'past_due'`. A user with `subscription_tier='basic'` but stale `'inactive' / 'canceled'` status was labelled FREE despite being a registered Basic subscriber.
- `src/app/api/stripe/webhook/route.ts` *never wrote* `trial_consumed_at` despite the migration existing. The flag was only ever set by the backfill, which excluded users without `stripe_subscription_id`. Without the flag, `TrialBanner` kept advertising "Try Pro free" forever.

Fix (three coordinated changes):

- Settings card label.** `src/components/app/Settings.tsx` derives `productTier` as `'basic' | 'pro' | null` (no longer defaults `null → 'basic'`). Badge label is now driven by `productTier` directly: *BASIC* for any user with `subscription_tier='basic'` regardless of status, *PRO* for `'pro'`, *FREE* only when nothing has ever been recorded. Body copy updated for the canceled/inactive Basic case so the user sees a clear "reactivate" message instead of a wrong "free" badge.
- Webhook stamps `trial_consumed_at`.** Both `handleCheckoutComplete` and `handleSubscriptionUpdate` in the webhook route now read `trial_consumed_at`, and stamp it once when the incoming subscription has `status='trialing'` or a non-null `trial_end`. Idempotent — only fires when the column is currently `NULL`.
- Banner suppression fallback.** `src/components/app/TrialBanner.tsx` adds a `hasStripeHistory` check (`!!plan || !!trialConsumed || status !== 'inactive'`). Even if `trial_consumed_at` is null on a legacy row that the migration didn't backfill, the banner self-suppresses for any user with a recorded subscription plan or a non-brand-new status. `isPaidBasic` also widened to include `past_due`.

Verification needed: three real-Clerk + real-Stripe scenarios — (1) paid Basic active user sees BASIC badge and no banner, (2) user whose trial ended sees no "Try Pro free" banner, (3) freshly-onboarded user with no `subscription_tier` still sees FREE + the trial offer. Cannot be exercised in demo mode (no real Stripe IDs). Migration `034_trial_consumed_at.sql` from Round 1 must still be applied.

BUG-031 (NEW Critical) — Stripe payments not reflected in Dyia

Root cause: The webhook only acted on `checkout.session.completed` with `metadata.payment_kind === 'customer_payment'`. Anything else (missing/different metadata, network reorder, or a Connect deployment with PI events only) silently fell through to `handleCheckoutComplete` (the subscription handler), which exits early on missing `session.subscription` — and the webhook still returned `200 OK`. Result: the merchant saw the money in Stripe but Dyia never updated the quote, the job, or the `/payments` view.

Fix (4 independent reconciliation paths so payments always reflect):

- **1. Hardened `checkout.session.completed` routing.** Now treats *any* of: `payment_kind === 'customer_payment'` OR a present `dyia_payment_id` OR a `payment-mode` session with `quote_id / job_id` as a customer payment. Subscription-mode sessions still use `handleCheckoutComplete`. Anything that doesn't match logs a loud `console.warn` with metadata keys so the next missed payment is debuggable from logs alone.
- **2. New `payment_intent.succeeded` handler.** `handlePaymentIntentSucceeded` resolves `dyia_payment_id` from the PI's metadata (we already attach it in `payment_intent_data.metadata` at session creation), or falls back to looking up by `stripe_payment_intent_id`. Runs the same DB updates as the session-based handler. Catches Connect destination-charges even if `checkout.session.completed` is misrouted, never delivered, or stripped of metadata. Idempotent — if already paid, no-op.
- **3. New `charge.refunded` handler.** Resolves the payment by PI id, marks `dyia_payments.status` as `'refunded'` (full) or `'partial_refund'`, and resets the linked quote/job `payment_status` to `'pending'` on full refund so the "Request Payment" CTA reappears. Unblocks STR-010 (refund reflection).
- **4. New `/api/payments/public/[token]/verify` route + auto-call from `/pay/[token]`.** When the customer is redirected back to `/pay/[token]?checkout=success&session_id=...`, the page now calls a small service-role endpoint that retrieves the Checkout Session from Stripe directly, confirms `payment_status === 'paid'`, and reconciles `dyia_payments + dyia_quotes + dyia_jobs` if the webhook hasn't yet. Sanity-guarded against forged sessions (rejects if `stripe_checkout_session_id` doesn't match). Guarantees the merchant sees an updated quote *even if the webhook is delayed or misconfigured*.

Code: `src/app/api/stripe/webhook/route.ts`,
`src/app/api/payments/public/[token]/verify/route.ts`, `src/app/pay/[token]/page.tsx`.

Verification needed: a real Stripe Connect account end-to-end. Once sign-off: STR-007 → STR-011 are unblocked, including refund reflection (STR-010). With three independent paths to reconcile a paid intent, the only way a payment can fail to reflect now is a total Stripe outage on both webhook delivery *and* the customer's redirect — extremely unlikely.

UX gap — Auto-copy payment link

Root cause: `navigator.clipboard.writeText(url)` was called *after* `await fetch(...)`. Browsers tie Clipboard API permissions to a transient user-activation window that expires across `await` boundaries — so the call rejected in practice on every Request Payment click, and the "couldn't copy automatically" alert ran every time. QA correctly perceived this as the default flow, not a fallback.

Fix: new component `src/components/app/PaymentLinkReadyModal.tsx`. After the link is created, an explicit modal opens with: a readonly `<input>` auto-selected on open (so `Cmd/Ctrl+C` works immediately), a primary "Copy link" button that does the clipboard write *synchronously inside the user-gesture handler* (succeeds reliably across Chromium/Safari/Firefox including iOS), an "Open link" button (new tab), and a Done button. Wired

into: `requestQuotePayment` in `Quotes.tsx`, `requestJobPayment` in `Jobs.tsx`, and the "Copy link" button in `Payments.tsx` (replaces the previous `window.prompt` fallback). The dialog now feels intentional, not like an error fallback.

Re-test of related test cases

Test ID	Action	Round 1 Result	Round 2 Expected
AI-002	Log a job via chat	Pass	Still Pass
AI-003	Update the just-logged job	Fail (BUG-011)	Pass — server-side context injection + recentJobs created_at order
AI-005	Follow-up customers list	Pass	Still Pass
AI-006	Top-earning jobs	Pass	Still Pass
AI-008	Create a quote via chat	Pass	Still Pass
STR-007	Initiate payment request	Pass	Pass — modal-based copy is reliable
STR-008	Complete a test payment	Pass	Pass
STR-009	Payment reflected in Jobs/Revenue	Fail (BUG-031)	Pass — webhook PI handler + verify route + hardened routing
STR-010	Refund via Stripe dashboard	Fail (BUG-031)	Pass — new charge.refunded handler resets quote/job to pending
STR-011	Payment flow on iOS / Android	Fail (BUG-031)	Pass — same server reconciliation; new modal works on mobile (synchronous click preserves user gesture on iOS Safari)

Verification coverage

The following were verified locally in this round:

- `npm run lint` — zero errors introduced (the 2 errors that remain in `Assistant.tsx` pre-exist and are unrelated to this batch).
- `npm run build` — clean. New route `/api/payments/public/[token]/verify` registers correctly.
- `POST /api/payments/public/<fake>/verify` returns 404 cleanly (no crash on unknown tokens).

- `/pay/[token]` renders the "Payment unavailable" fallback for unknown tokens; mobile viewport (390x844) clean — no layout regressions on the marketing or payment pages.
- Code review of every changed file with `ReadLints` — only pre-existing Tailwind v4 stylistic warnings remain (313 such warnings, all pre-existing in the codebase, not introduced here).

The following ship as code fixes and require human QA on real Clerk / Stripe / iOS sessions:

- BUG-011 — needs an authenticated AI session to confirm the round-trip behaviour.
- BUG-022 — needs a real Basic subscriber, a user with consumed trial, and a brand-new no-subscription user to verify the badge + banner branches.
- BUG-031 — needs a real Stripe Connect account, a real test card, and a refund through Stripe Dashboard to verify all three reconciliation paths.
- UX gap — needs a manual click on iPhone Safari + Android Chrome to confirm the synchronous Copy button succeeds in real browser sessions.

Required actions before deploy

Migrations

Round 1's `supabase/migrations/034_trial_consumed_at.sql` must already be applied. No new migration is needed in Round 2 — the new code reads/writes the same columns.

Stripe webhook configuration

Make sure the platform Stripe webhook endpoint is subscribed to *all* of: `checkout.session.completed`, `payment_intent.succeeded`, `charge.refunded`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.paid`, `invoice.payment_failed`. The first three are required for customer-payment reflection (BUG-031); the others were already wired and remain unchanged.

File-by-file summary of changes

File	Change	Bug
<code>src/app/globals.css</code>	<code>viewEnter</code> ends at <code>transform: none</code> ; <code>bannerSlideDown</code> ends at <code>max-height: 16rem</code>	BUG-016, BUG-018
<code>src/components/app/Jobs.tsx</code>	Sort/source selects gain <code>min-w</code> ; payment link modal wired in	BUG-015,

File	Change	Bug
		UX gap
src/components/app/TopBar.tsx	Mobile logo conditionally swaps to /dyia-logo-dark.png in dark mode; dark filters removed	BUG-020
src/components/app/Settings.tsx	Plan badge derived from productTier (BASIC vs PRO vs FREE) regardless of subscription status	BUG-022
src/components/app/TrialBanner.tsx	Suppresses banner when any Stripe history exists; widened paid-Basic check	BUG-022
src/app/api/stripe/webhook/route.ts	Stamps trial_consumed_at; hardened checkout routing; new payment_intent.succeeded + charge.refunded handlers	BUG-022, BUG-031
src/app/api/payments/public/[token]/verify/route.ts	NEW — service-role reconciliation endpoint	BUG-031
src/app/pay/[token]/page.tsx	Calls verify route after Stripe redirect; shows "Confirming your payment..." state	BUG-031
src/components/app/Quotes.tsx	Replaces post-await clipboard flow with payment link modal	UX gap
src/components/app/Payments.tsx	Replaces window.prompt fallback with payment link modal	UX gap
src/components/app/PaymentLinkReadyModal.tsx	NEW — explicit copy modal with synchronous Copy button	UX gap
src/components/app/Assistant.tsx	lastConfirmedRef capture + injection on next chat call	BUG-011
src/app/api/ai/chat/route.ts	Server-side [CONTEXT] preamble injection from lastConfirmed	BUG-011
src/lib/openai/handlers.ts	recentJobs ordered by created_at desc	BUG-011

Out of scope

- iOS Safari combobox / tap-target verification — requires real device, flagged for QA.
- Adding a separate Stripe Connect-account webhook endpoint — not needed; destination charges with platform secret emit on the platform endpoint already, and the new `payment_intent.succeeded` handler covers the case where ops accidentally subscribe only PI events.
- Refactoring the pre-existing 2 `react-hooks/set-state-in-effect` errors in `Assistant.tsx` — not introduced in this batch, no behaviour change.